

Assignment 1: Transformers from Scratch, Architectural Variants, and Byte Latent Transformers

Dhyey Thummar - 2024101024
Advanced Natural Language Processing

Abstract

This work studies five encoder–decoder Transformer configurations for mapping encrypted binary sequences back to plaintext. The baseline uses sinusoidal positional encoding, multi-head attention (MHA), LayerNorm, and learned BPE tokenization. Three controlled variants replace one component at a time with RoPE, grouped-query attention (GQA), or RMSNorm. The fifth configuration replaces learned subword tokenization with a token-free byte-level pipeline using local 4-byte patches. All models were implemented in PyTorch without `nn.Transformer` or `nn.MultiheadAttention` and were trained with the same core hyperparameters. On the primary chunk-level test protocol, the baseline reached 92.51% bit accuracy, while the BLT configuration reached 100.00% bit accuracy and 99.91% exact sequence accuracy. BLT also reduced peak GPU memory from 1166 MB to 420 MB.

1 Experimental Setup

1.1 Data and preprocessing

The dataset contains paired binary ciphertext and plaintext lines. The original lines were split 80/10/10 into training, validation, and test sets before chunking. Each plaintext line was then divided into non-overlapping 64-character ASCII chunks. A full 64-character chunk corresponds to 512 ciphertext bits, and the final shorter chunk of a line was retained. This produced 38,257 training chunks, 5,406 validation chunks, and 5,487 test chunks.

For C1–C4, two custom BPE tokenizers were trained from scratch. The ciphertext tokenizer learns directly over the binary characters 0 and 1 with a vocabulary size of 2048. The plaintext tokenizer uses a vocabulary size of 4096. On validation chunks, the ciphertext BPE achieved 13.06× compression and the plaintext BPE achieved 3.64× compression, with 0% unknown tokens.

C5 does not use learned subword tokens. Every 8 ciphertext bits are converted to one raw byte, and local modules group four bytes into one patch representation.

The primary evaluation treats every 64-character chunk as one independent sequence sample. All outputs are generated using greedy decoding. Bit accuracy is computed from the 8-bit ASCII representations of the reference and prediction.

The implementation also reconstructs full original test lines as a secondary diagnostic, but all C1–C5 comparisons in this report use the same chunk-level evaluation protocol.

1.2 Model and training settings

All configurations use $d_{model} = 256$, 8 attention heads, 4 encoder layers, 4 decoder layers, a feed-forward dimension of 1024, dropout 0.1, and maximum sequence length 1024.

C3 uses 8 query heads and 2 key/value heads for GQA. C5 uses 4-byte patches with a lightweight local Transformer before and after the global Transformer.

Training used AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.98$, weight decay 0.01, batch size 64, label smoothing 0.05, and gradient clipping at 1.0.

Each run used 25,000 optimizer steps. The learning rate linearly warmed up to 3×10^{-4} over 2,000 steps and then followed cosine decay to 3×10^{-5} .

Mixed-precision FP16 training was used on an NVIDIA GeForce RTX 3050 Laptop GPU. The random seed was 42. Training and validation statistics were logged to Weights & Biases.

2 Architectural Ablations

The baseline follows the encoder–decoder Transformer design [1]. The ablations use rotary position embeddings [2], grouped-query attention [3], RMSNorm [4], and a simplified byte-latent design inspired by BLT [5].

Table 1: Controlled configurations. Each variant changes one main component from C1.

Config	Positional encoding	Attention	Normalization	Input representation
C1	Sinusoidal absolute	MHA	LayerNorm	Learned BPE
C2	RoPE	MHA	LayerNorm	Learned BPE
C3	Sinusoidal absolute	GQA	LayerNorm	Learned BPE
C4	Sinusoidal absolute	MHA	RMSNorm	Learned BPE
C5	Sinusoidal absolute	MHA	LayerNorm	Raw bytes + 4-byte local patches

3 Results :-

Table 2: Primary chunk-level greedy-decoding results on 5,487 test chunks. Higher is better for accuracy, BLEU and ROUGE-L; lower is better for Levenshtein distance. Following the assignment specification, BLEU and ROUGE-L are reported only for the tokenized C1–C4 configurations; C5 is token-free.

Config	Seq. Acc. (%)	Bit Acc. (%)	Lev. ↓	BLEU	ROUGE-L (%)
C1	46.75	92.51	1.78	81.93	92.03
C2	45.47	92.65	1.83	81.20	91.75
C3	43.12	92.35	2.03	80.13	91.13
C4	46.07	92.90	1.76	81.70	91.89
C5	99.91	100.00	0.00	N/A	N/A

Table 3: Training and validation efficiency. All configurations were trained for 25,000 optimizer steps.

Config	Time (min)	Peak GPU (MB)	Best val. loss	Val. token acc. (%)
C1	31.1	1166	1.0479	91.33
C2	32.9	1171	1.0527	91.31
C3	25.4	1153	1.0921	90.41
C4	23.2	1084	1.0555	91.08
C5	27.9	420	0.4865	100.00

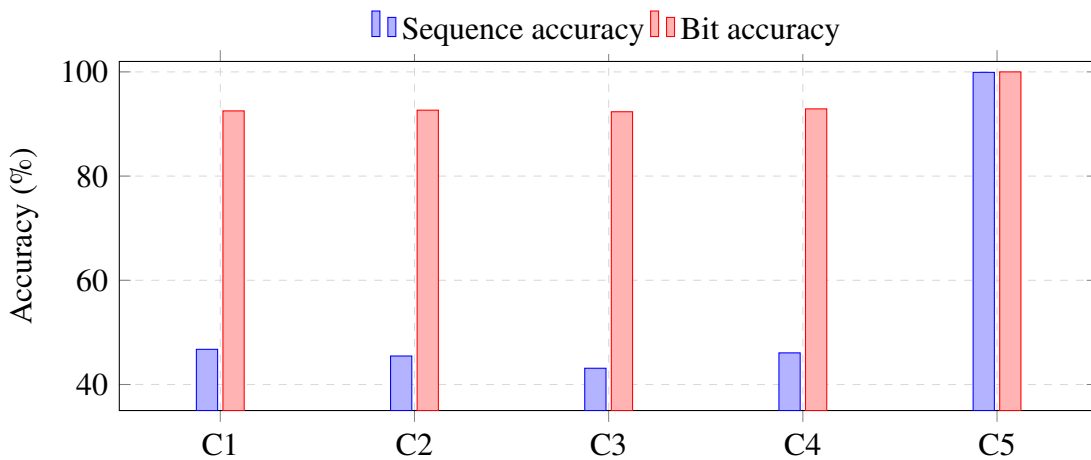


Figure 1: Chunk-level exact sequence accuracy and bit-level accuracy across C1–C5.

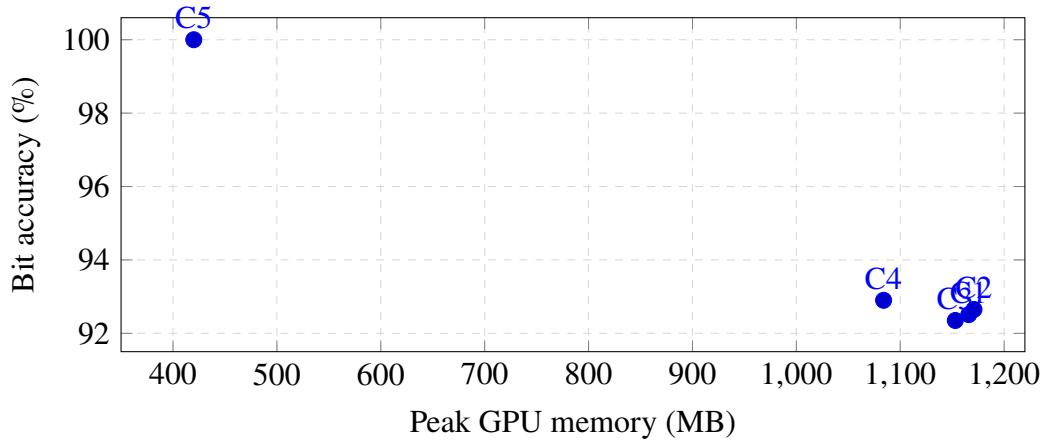


Figure 2: Accuracy–memory tradeoff. C5 gives the highest bit accuracy while using much less peak GPU memory.

4 Discussion and Comparison :-

C2: RoPE vs. sinusoidal encoding. Replacing sinusoidal encoding with RoPE produced almost the same overall behavior. Bit accuracy increased slightly from 92.51% to 92.65%, a gain of 0.13 percentage points, but exact sequence accuracy decreased from 46.75% to 45.47%. BLEU and ROUGE-L also decreased slightly, and training time increased from 31.1 to 32.9 minutes. For this task and sequence length, RoPE therefore did not provide a clear advantage over sinusoidal encoding.

C3: GQA vs. MHA. GQA gave the weakest tokenized reconstruction results. Sequence accuracy dropped by 3.63 percentage points relative to C1 and the mean Levenshtein distance increased from 1.78 to 2.03. Bit accuracy changed only slightly, from 92.51% to 92.35%, showing that many outputs were still close to the target even when they were not exact. The run was faster than C1, taking 25.4 minutes instead of 31.1 minutes. GQA therefore provided a speed benefit but with a small loss in reconstruction quality.

C4: RMSNorm vs. LayerNorm. RMSNorm was the strongest tokenized variant in bit accuracy, reaching 92.90%, which is 0.39 percentage points above C1. It also achieved the lowest tokenized Levenshtein distance of 1.76. Exact sequence accuracy was slightly lower than C1 (46.07% vs. 46.75%), so the gain was mainly in average reconstruction quality rather than exact chunk matches. C4 also had the shortest measured training time, 23.2 minutes, and used less peak GPU memory than C1.

C5: token-free BLT vs. BPE. C5 produced the largest change. It achieved 99.91% exact sequence accuracy and 100.00% bit accuracy, compared with 46.75% and 92.51% for C1.

Peak GPU memory dropped from 1166 MB to 420 MB, a reduction of about 64%, and total training time decreased from 31.1 to 27.9 minutes.

In this implementation, the local byte modules therefore did not create a net training-time penalty. The 4-byte patching scheme reduces the sequence length seen by the global Transformer and avoids the large learned source and target subword vocabularies.

On this byte-aligned deterministic mapping, the byte-level representation matched the data especially well. BLEU and ROUGE-L are marked N/A for C5 because the assignment specifies these metrics only for tokenized models. C5 is token-free, so the main comparison for C5 uses bit accuracy, sequence accuracy, and Levenshtein distance.

Across all configurations, source-shuffling diagnostics caused a large increase in validation loss. This indicates that the models were using the ciphertext input rather than relying only on plaintext language patterns.

Overall, C5 gave the strongest result, while C4 was the strongest BPE-based configuration in terms of bit accuracy and efficiency.

5 Conclusion :-

The ablation study shows that changing RoPE, GQA, or RMSNorm alone caused only small differences around the BPE baseline. RMSNorm gave the best tokenized bit accuracy, while GQA trained faster but reduced exact reconstruction quality.

The main improvement came from the token-free BLT design. Raw-byte processing with local 4-byte patches achieved nearly perfect reconstruction while using much less GPU memory.

For this dataset, the input representation had a much larger effect than the other single architectural changes.

Links :-

Individual WandB runs: [C1](#) [C2](#) [C3](#) [C4](#) [C5](#)

Hugging Face checkpoints:

<https://huggingface.co/dhyeythummar9/ANLP-A1-Transformer-Ablations>

References

- [1] A. Vaswani et al., “Attention Is All You Need,” *NeurIPS*, 2017.
- [2] J. Su et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” 2021.
- [3] J. Ainslie et al., “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” 2023.
- [4] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” *NeurIPS*, 2019.
- [5] A. Pagnoni et al., “Byte Latent Transformer: Patches Scale Better Than Tokens,” 2024.